

```

<?php
// pages/administrar.php
require_once '../config.php';

// Validar que el usuario logueado sea estrictamente ADMIN
if (!isset($_SESSION['usuario']) || strtoupper($_SESSION['usuario']['rol'] ?? '')
!= 'ADMIN') {
    header('Location: ../login.php');
    exit;
}
?>
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Banco Escolar - Administración General</title>
    <link rel="stylesheet" href="../css/cajero.css">
</head>
<body>

    <!-- NAVBAR DESDE SERVIDOR -->
    <?php include __DIR__ . '/../includes/navbar.php'; ?>

    <main class="dashboard-container">
        <div class="page-header">
            <h1>Panel de Administración General</h1>
            <p class="text-subtitle">Gestión de accesos, terminales POSNET, cursos,
importación de datos y mantenimiento.</p>
        </div>

        <div class="admin-grid">
            <!-- SECCIÓN 1: GESTIÓN DE USUARIOS BANCO (Cajeros y Admins) -->
            <section class="card">
                <h3>👤 Usuarios del Sistema</h3>
                <p class="text-subtitle">Crear/Editar cajeros y administradores</p>
                <form id="formUsuarioBanco" style="margin-bottom: 20px;">
                    <input type="hidden" id="usrId">
                    <div class="form-group">
                        <label for="txtUsrUsuario">Usuario / Login</label>
                        <input type="text" id="txtUsrUsuario" placeholder="Ej:
cajero_turno1" required>
                    </div>
                    <div class="form-group">
                        <label for="txtUsrNombre">Nombre Completo</label>
                        <input type="text" id="txtUsrNombre" placeholder="Ej: María
López" required>
                    </div>
                    <div class="form-group">
                        <label for="txtUsrPass">Contraseña / Password</label>

```

```

        <input type="password" id="txtUsrPass"
placeholder="..... (dejar en blanco para no cambiar)">
    </div>
    <div class="form-group">
        <label for="selUsrRol">Rol de Sistema</label>
        <select id="selUsrRol">
            <option value="CAJERO">CAJERO</option>
            <option value="ADMIN">ADMINISTRADOR</option>
        </select>
    </div>
    <div class="form-group" style="display: flex; gap: 8px;
align-items: center;">
        <input type="checkbox" id="chkPuedeRetirar" style="width:
auto;">
        <label for="chkPuedeRetirar" style="margin: 0; font-weight:
normal;">¿Permite Retiro de Efectivo?</label>
    </div>
    <div style="display: flex; gap: 8px; margin-top: 12px;">
        <button type="submit" id="btnGuardarUsr" class="btn
btn-primary" style="flex: 1;">Crear Usuario</button>
        <button type="button" id="btnCancelarUsr" class="btn
btn-dark" style="display: none;">Cancelar</button>
    </div>
</form>
<div id="listaUsuariosBanco"></div>
</section>

<!-- SECCIÓN 2: CONTROL DE TERMINALES POSNET -->
<section class="card">
    <h3>☰ POSNETS / Terminales de Cobro</h3>
    <p class="text-subtitle">Alta, edición y activación de puntos de
cobro</p>
    <form id="formPosnet" style="margin-bottom: 20px;">
        <input type="hidden" id="posnetId">
        <div class="form-group">
            <label for="txtPosnetNombre">Nombre del Stand /
Puesto</label>
            <input type="text" id="txtPosnetNombre" placeholder="Ej:
Stand Buffet 5to Año" required>
        </div>
        <div class="form-group">
            <label for="txtPosnetUsuario">Usuario de Acceso
POSNET</label>
            <input type="text" id="txtPosnetUsuario" placeholder="Ej:
posnet_buffet" required>
        </div>
        <div class="form-group">
            <label for="txtPosnetPass">Contraseña POSNET</label>
            <input type="password" id="txtPosnetPass"
placeholder="..... (dejar en blanco para mantener)">

```

```

        </div>
        <div style="display: flex; gap: 8px; margin-top: 12px;">
            <button type="submit" id="btnGuardarPosnet" class="btn
btn-primary" style="flex: 1;">Registrar Terminal</button>
            <button type="button" id="btnCancelarPosnet" class="btn
btn-dark" style="display: none;">Cancelar</button>
        </div>
    </form>
    <div id="listaPosnets"></div>
</section>

<!-- SECCIÓN 3: CRUD DE CURSOS / DIVISIONES -->
<section class="card">
    <h3>📁 Cursos y Divisiones</h3>
    <p class="text-subtitle">Precarga de años/cursos para el alta de
alumnos</p>
    <form id="formCurso" style="margin-bottom: 20px; display: flex;
gap: 10px;">
        <input type="text" id="txtNombreCurso" placeholder="Ej: 6to 1ra
Informática" style="flex: 1;" required>
        <button type="submit" class="btn btn-primary">Agregar</button>
    </form>
    <div style="overflow-x: auto;">
        <table class="data-table">
            <thead>
                <tr>
                    <th>Nombre del Curso</th>
                    <th style="text-align: right;">Acción</th>
                </tr>
            </thead>
            <tbody id="tblCursos"></tbody>
        </table>
    </div>
</section>

<!-- SECCIÓN 4: IMPORTAR ALUMNOS DESDE CSV -->
<section class="card">
    <h3>📁 Importar Alumnos (CSV)</h3>
    <p class="text-subtitle">Carga masiva para agilizar el alta.
Formato: DNI, Nombre, Curso, Saldo, Foto URL, QR</p>
    <form id="formImportarCSV" enctype="multipart/form-data">
        <div class="form-group">
            <label for="fileCsv">Seleccionar archivo .csv</label>
            <input type="file" id="fileCsv" accept=".csv" required
style="margin-top: 5px;">
        </div>
        <button type="submit" class="btn btn-primary" style="width:
100%; margin-top: 10px;">Cargar Datos CSV</button>
    </form>
</section>

```

```

<!-- SECCIÓN 5: MANTENIMIENTO Y RESET DE BASE DE DATOS -->
<section class="card card-extraccion">
  <h3 style="color: #e53e3e;">⚠ Mantenimiento y Reset BD</h3>
  <p class="text-subtitle">Seleccioná los datos a reiniciar. Requiere
autorización con QR Maestro.</p>
  <div class="form-group" style="gap: 10px; margin: 15px 0;">
    <label style="font-weight: normal;"><input type="checkbox"
id="chkResetMontos"> Resetear montos acumulados a $0 (Cajeros y Posnets)</label>
    <label style="font-weight: normal;"><input type="checkbox"
id="chkVaciarTransacciones"> Vaciar historial de Transacciones</label>
    <label style="font-weight: normal;"><input type="checkbox"
id="chkVaciarLogs"> Vaciar Logs de Sistema y Auditoría</label>
    <label style="font-weight: normal;"><input type="checkbox"
id="chkBorrarNoReg"> Eliminar alumnos NO registrados</label>
    <label style="font-weight: bold; color: #c53030;"><input
type="checkbox" id="chkVaciarAlumnos" style="accent-color: #e53e3e;"> Vaciar TODOS
los Alumnos</label>
  </div>
  <button type="button" id="btnProcesarReset" class="btn
btn-danger">Ejecutar Mantenimiento</button>
</section>
</div>
</main>

<!-- MODAL DE CONFIRMACIÓN CON CÁMARA QR PARA RESET -->
<div id="modalResetQR" class="modal-qr-overlay" style="display: none;">
  <div class="modal-qr-content">
    <h3 style="color: #fc8181;">Autorizar Mantenimiento</h3>
    <p style="font-size: 0.85rem; color: #a0aec0; margin-bottom:
15px;">Escaneá el QR Maestro para aplicar los cambios en la BD.</p>
    <div id="readerReset" style="width: 100%; max-width: 300px; margin: 0
auto;"></div>
    <button type="button" id="btnCerrarResetQR" class="btn btn-dark"
style="margin-top: 15px; width: 100%;">Cancelar</button>
  </div>
</div>

<!-- MODAL DE CONFIRMACIÓN PARA ELIMINAR -->
<div id="modalEliminar" class="modal-backdrop" style="display: none;">
  <div class="modal-box">
    <h3>⚠ Confirmar Eliminación</h3>
    <p id="msgConfirmarEliminar">¿Estás seguro de que deseas eliminar este
registro?</p>
    <div class="modal-actions">
      <button id="btnConfirmarEliminar" class="btn btn-danger">Sí,
Eliminar</button>
      <button onclick="cerrarModalEliminar()" class="btn
btn-dark">Cancelar</button>
    </div>
  </div>

```

```

        </div>
    </div>

    <script src="https://unpkg.com/html5-qrcode"></script>
    <script src="../../js/administrar.js"></script>
</body>
</html>

let idEliminarPendiente = null;

// Funciones globales para que onclick pueda encontrarlas
window.abrirModalEliminar = function(id, nombre) {
    idEliminarPendiente = id;
    const msg = document.getElementById('msgConfirmarEliminar');
    if (msg) msg.innerText = `¿Estás seguro de que deseas eliminar "${nombre}"?
Esta acción no se puede deshacer.`;
    const modal = document.getElementById('modalEliminar');
    if (modal) modal.style.display = 'flex';
};

window.cerrarModalEliminar = function() {
    idEliminarPendiente = null;
    const modal = document.getElementById('modalEliminar');
    if (modal) modal.style.display = 'none';
};

document.addEventListener('DOMContentLoaded', () => {
    let qrResetScanner = null;
    let cacheUsuarios = [];
    let cachePosnets = [];

    // Cargar datos iniciales
    cargarUsuarios();
    cargarPosnets();
    cargarCursos();

    // =====
    // 1. USUARIOS DEL BANCO
    // =====
    async function cargarUsuarios() {
        try {
            const res = await
fetch('../../procesar_admin.php?action=listar_usuarios');
            const data = await res.json();
            if (!data.success) return;

            cacheUsuarios = Array.isArray(data.data) ? data.data : [];
            const cont = document.getElementById('listaUsuariosBanco');
            if (!cont) return;

```

```

        if (cacheUsuarios.length === 0) {
            cont.innerHTML = `

No hay usuarios registrados.</p>`;
            return;
        }

        cont.innerHTML = cacheUsuarios.map(u => `
            <div class="item-lista">
                <div class="item-info">
                    <h4>${u.usuario} <span class="item-badge badge-${(u.rol ||
''.toLowerCase())}>${u.rol}</span></h4>
                    <p>${u.nombre} | Retiro: ${u.puede_retirar ? 'Sí' :
'NO'}</p>
                </div>
                <div class="item-acciones">
                    <button onclick="editarUsuario('${u.id}')" class="btn
btn-dark btn-sm">Editar</button>
                    <button onclick="toggleEstadoUsuario('${u.id}',
${!u.activo})" class="btn ${u.activo ? 'btn-danger' : 'btn-success'} btn-sm">
                        ${u.activo ? 'Desactivar' : 'Activar'}
                    </button>
                    <button onclick="abrirModalEliminar('${u.id}',
`${u.usuario}')" class="btn btn-danger btn-sm" style="background:#c53030;">🗑️
Eliminar</button>
                </div>
            </div>
        `).join('');
    } catch (e) { console.error("Error al cargar usuarios:", e); }
}


```

```

window.editarUsuario = (id) => {
    const usr = cacheUsuarios.find(u => u.id == id);
    if (!usr) return;
    document.getElementById('usrId').value = usr.id;
    document.getElementById('txtUsrUsuario').value = usr.usuario;
    document.getElementById('txtUsrNombre').value = usr.nombre;
    document.getElementById('txtUsrPass').value = '';
    document.getElementById('selUsrRol').value = usr.rol;
    document.getElementById('chkPuedeRetirar').checked = !!usr.puede_retirar;
    document.getElementById('btnGuardarUsr').textContent = 'Actualizar
Usuario';
    document.getElementById('btnCancelarUsr').style.display = 'inline-block';
};

```

```

document.getElementById('btnCancelarUsr')?.addEventListener('click', () => {
    document.getElementById('formUsuarioBanco').reset();
    document.getElementById('usrId').value = '';
    document.getElementById('btnGuardarUsr').textContent = 'Crear Usuario';
    document.getElementById('btnCancelarUsr').style.display = 'none';
});

```

```

document.getElementById('formUsuarioBanco')?.addEventListener('submit', async
(e) => {
    e.preventDefault();
    const payload = {
        id: document.getElementById('usrId').value,
        usuario: document.getElementById('txtUsrUsuario').value,
        nombre: document.getElementById('txtUsrNombre').value,
        password: document.getElementById('txtUsrPass').value,
        rol: document.getElementById('selUsrRol').value,
        puede_retirar: document.getElementById('chkPuedeRetirar').checked
    };

    const res = await fetch('../procesar_admin.php?action=guardar_usuario', {
        method: 'POST',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify(payload)
    });
    const data = await res.json();
    alert(data.message);
    if (data.success) {
        e.target.reset();
        document.getElementById('usrId').value = '';
        document.getElementById('btnGuardarUsr').textContent = 'Crear Usuario';
        document.getElementById('btnCancelarUsr').style.display = 'none';
        cargarUsuarios();
    }
});

window.toggleEstadoUsuario = async (id, nuevoEstado) => {
    await fetch('../procesar_admin.php?action=cambiar_estado_usuario', {
        method: 'POST',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify({ id, activo: nuevoEstado })
    });
    cargarUsuarios();
};

// Evento del botón de confirmación del Modal de Eliminación
document.getElementById('btnConfirmarEliminar')?.addEventListener('click',
async () => {
    if (!idEliminarPendiente) return;
    try {
        const res = await
fetch('../procesar_admin.php?action=eliminar_usuario', {
            method: 'POST',
            headers: {'Content-Type': 'application/json'},
            body: JSON.stringify({ id: idEliminarPendiente })
        });
        const data = await res.json();

```

```

        cerrarModalEliminar();
        alert(data.message);
        if (data.success) {
            cargarUsuarios();
            cargarPosnets();
        }
    } catch (e) {
        console.error("Error al eliminar:", e);
        cerrarModalEliminar();
    }
});

// =====
// 2. TERMINALES POSNET
// =====
async function cargarPosnets() {
    try {
        const res = await fetch('../procesar_admin.php?action=listar_posnets');
        const data = await res.json();
        if (!data.success) return;

        cachePosnets = Array.isArray(data.data) ? data.data : [];
        const cont = document.getElementById('listaPosnets');
        if (!cont) return;

        if (cachePosnets.length === 0) {
            cont.innerHTML = `

No hay terminales POSNET registradas.</p>`;
            return;
        }

        cont.innerHTML = cachePosnets.map(p => `
            <div class="item-lista">
                <div class="item-info">
                    <h4>${p.nombre}</h4>
                    <p>Usuario: <b>${p.usuario}</b> | Acumulado:
                    ${parseFloat(p.monto_acumulado || 0).toFixed(2)}</p>
                </div>
                <div class="item-acciones">
                    <button onclick="editarPosnet('${p.id}')" class="btn btn-dark btn-sm">Editar</button>
                    <button onclick="toggleEstadoPosnet('${p.id}',
                    ${!p.activo})" class="btn ${p.activo ? 'btn-danger' : 'btn-success'} btn-sm">
                        ${p.activo ? 'Baja' : 'Alta'}
                    </button>
                    <button onclick="abrirModalEliminar('${p.id}',
                    '${p.nombre}')" class="btn btn-danger btn-sm" style="background:#c53030;">
                    Eliminar</button>
                </div>
            `
    )


```



```

        </div>
    `).join('');
} catch (e) { console.error("Error al cargar posnets:", e); }
}

window.editarPosnet = (id) => {
    const pos = cachePosnets.find(p => p.id == id);
    if (!pos) return;
    document.getElementById('posnetId').value = pos.id;
    document.getElementById('txtPosnetNombre').value = pos.nombre;
    document.getElementById('txtPosnetUsuario').value = pos.usuario;
    document.getElementById('txtPosnetPass').value = '';
    document.getElementById('btnGuardarPosnet').textContent = 'Actualizar
Terminal';
    document.getElementById('btnCancelarPosnet').style.display =
'inline-block';
};

document.getElementById('btnCancelarPosnet')?.addEventListener('click', () => {
    document.getElementById('formPosnet').reset();
    document.getElementById('posnetId').value = '';
    document.getElementById('btnGuardarPosnet').textContent = 'Registrar
Terminal';
    document.getElementById('btnCancelarPosnet').style.display = 'none';
});

document.getElementById('formPosnet')?.addEventListener('submit', async (e) =>
{
    e.preventDefault();
    const idInput = document.getElementById('posnetId').value;
    const payload = {
        id: idInput.trim() !== '' ? idInput.trim() : null,
        nombre_stand: document.getElementById('txtPosnetNombre').value,
        usuario_posnet: document.getElementById('txtPosnetUsuario').value,
        password_posnet: document.getElementById('txtPosnetPass').value
    };

    try {
        const res = await fetch('../procesar_admin.php?action=guardar_posnet',
{
            method: 'POST',
            headers: {'Content-Type': 'application/json'},
            body: JSON.stringify(payload)
        });

        if (!res.ok) {
            throw new Error("Error en la petición: Status HTTP " + res.status);
        }

        const data = await res.json();

```

```

        alert(data.message);
        if (data.success) {
            document.getElementById('formPosnet').reset();
            document.getElementById('posnetId').value = '';
            document.getElementById('btnGuardarPosnet').textContent =
'Registrar Terminal';
            document.getElementById('btnCancelarPosnet').style.display =
'none';
            cargarPosnets();
        }
    } catch (err) {
        console.error("Error al guardar posnet", err);
        alert("Ocurrió un error al procesar la solicitud.");
    }
});

window.toggleEstadoPosnet = async (id, nuevoEstado) => {
    try {
        await fetch('../procesar_admin.php?action=cambiar_estado_posnet', {
            method: 'POST',
            headers: {'Content-Type': 'application/json'},
            body: JSON.stringify({ id, activo: nuevoEstado })
        });
        cargarPosnets();
    } catch (e) {
        console.error("Error al cambiar estado:", e);
    }
};

// =====
// 3. CURSOS / DIVISIONES
// =====
async function cargarCursos() {
    try {
        const res = await fetch('../procesar_admin.php?action=listar_cursos');
        const data = await res.json();
        if (!data.success) return;

        const cursosLista = Array.isArray(data.data) ? data.data : [];
        const tbody = document.getElementById('tblCursos');
        if (!tbody) return;

        if (cursosLista.length === 0) {
            tbody.innerHTML = `<tr><td colspan="2" style="text-align:center;
color: #718096;">No hay cursos registrados.</td></tr>`;
            return;
        }

        tbody.innerHTML = cursosLista.map(c => `
            <tr>

```

```

        <td>${c.nombre}</td>
        <td style="text-align: right;">
            <button onclick="eliminarCurso('${c.id}')" class="btn
btn-danger" style="padding: 2px 8px; font-size: 0.75rem;">Eliminar</button>
        </td>
    </tr>
    `).join('');
    } catch (e) { console.error("Error al cargar cursos:", e); }
}

document.getElementById('formCurso')?.addEventListener('submit', async (e) => {
    e.preventDefault();
    const payload = { nombre: document.getElementById('txtNombreCurso').value
};

    const res = await fetch('../procesar_admin.php?action=guardar_curso', {
        method: 'POST',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify(payload)
    });
    const data = await res.json();
    alert(data.message);
    if (data.success) {
        e.target.reset();
        cargarCursos();
    }
});

window.eliminarCurso = async (id) => {
    if (!confirm("¿Seguro de eliminar este curso?")) return;
    await fetch('../procesar_admin.php?action=eliminar_curso', {
        method: 'POST',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify({ id })
    });
    cargarCursos();
};

// =====
// 4. IMPORTAR CSV
// =====
document.getElementById('formImportarCSV')?.addEventListener('submit', async (e) =>
{
    e.preventDefault(); // Evita que la página se recargue

    const fileInput = document.getElementById('fileCsv');
    if (!fileInput || !fileInput.files.length) {
        alert("Por favor, seleccioná un archivo CSV.");
        return;
    }
}

```

```

const formData = new FormData();
formData.append('fileCsv', fileInput.files[0]);

try {
  const res = await fetch('../procesar_admin.php?action=importar_csv', {
    method: 'POST',
    body: formData
    // IMPORTANTE: NO agregar 'Content-Type': 'multipart/form-data' acá.
    // El navegador debe establecer el boundary automáticamente.
  });

  if (!res.ok) {
    throw new Error(`Error HTTP Status: ${res.status}`);
  }

  const data = await res.json();
  alert(data.message);
  if (data.success) fileInput.value = '';
} catch (err) {
  // Imprimir el error exacto en la consola para depurar
  console.error("Error detallado en la importación:", err);
  alert("Error al procesar el archivo CSV. Revisa la consola (F12) para más
detalles.");
}
});

// =====
// 5. RESET Y MANTENIMIENTO CON CÁMARA QR
// =====
document.getElementById('btnProcesarReset')?.addEventListener('click', () => {
  const opciones = getOpcionesReset();
  if (!Object.values(opciones).some(v => v)) {
    alert("Seleccioná al menos una opción para el mantenimiento.");
    return;
  }

  document.getElementById('modalResetQR').style.display = 'flex';
  if (!qrResetScanner) {
    qrResetScanner = new Html5QrcodeScanner("readerReset", { fps: 10,
qrbox: 200 }, false);
  }
  qrResetScanner.render(onScanResetSuccess, () => {});
});

document.getElementById('btnCerrarResetQR')?.addEventListener('click',
cerrarResetQR);

function cerrarResetQR() {
  if (qrResetScanner) qrResetScanner.clear().catch(() => {});
}

```

```

        document.getElementById('modalResetQR').style.display = 'none';
    }

    async function onScanResetSuccess(qrMaestroText) {
        cerrarResetQR();
        const payload = {
            action: 'reset_bd',
            qr_code: qrMaestroText,
            opciones: getOpcionesReset()
        };

        try {
            const res = await fetch('../procesar_admin.php', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify(payload)
            });
            const data = await res.json();
            alert(data.message);
            if (data.success) location.reload();
        } catch (err) { alert("Error al ejecutar el mantenimiento."); }
    }

    function getOpcionesReset() {
        return {
            reset_acumulados: document.getElementById('chkResetMontos').checked,
            vaciar_transacciones:
document.getElementById('chkVaciarTransacciones').checked,
            vaciar_logs: document.getElementById('chkVaciarLogs').checked,
            borrar_alumnos_no_registrados:
document.getElementById('chkBorrarNoReg').checked,
            vaciar_alumnos_todos:
document.getElementById('chkVaciarAlumnos').checked
        };
    }
});

<?php
// procesar_admin.php
header('Content-Type: application/json');
require_once __DIR__ . '/config.php';

// Validar que el usuario esté autenticado y sea ADMIN
if (!isset($_SESSION['usuario']) || strtoupper($_SESSION['usuario']['rol'] ?? '')
!== 'ADMIN') {
    echo json_encode(['success' => false, 'message' => 'Acceso no autorizado.']);
    exit;
}

$input = json_decode(file_get_contents('php://input'), true) ?? [];

```

```

$action = $_GET['action'] ?? ($input['action'] ?? '');

try {
    switch ($action) {
        // =====
        // 1. GESTIÓN DE USUARIOS DEL BANCO
        // =====
        case 'listar_usuarios':
            $res =
supabaseQuery("usuarios_banco?rol=neq.POSNET&select=*&order=id.asc", "GET");
            echo json_encode(['success' => true, 'data' => is_array($res) ? $res :
[]]);
            break;

        case 'guardar_usuario':
            $id = $input['id'] ?? null;
            $payload = [
                'usuario'      => trim($input['usuario'] ?? ''),
                'nombre'       => trim($input['nombre'] ?? ''),
                'rol'          => strtoupper(trim($input['rol'] ?? 'CAJERO')),
                'puede_retirar' => !empty($input['puede_retirar']),
                'activo'       => true
            ];

            // Mapeo corregido a password_hash
            if (!empty($input['password'])) {
                $payload['password_hash'] = trim($input['password']);
            }

            if ($id) {
                $res = supabaseQuery("usuarios_banco?id=eq." . urlencode($id),
"PATCH", $payload);
            } else {
                $payload['monto_acumulado'] = 0;
                $payload['cant_transacciones'] = 0;
                $res = supabaseQuery("usuarios_banco", "POST", $payload);
            }

            if (isset($res['message']) || isset($res['error'])) {
                $errorMsg = $res['message'] ?? $res['error'] ?? 'Error
desconocido';
                echo json_encode(['success' => false, 'message' => 'Error de
Supabase: ' . $errorMsg]);
                exit;
            }

            echo json_encode(['success' => true, 'message' => 'Usuario guardado
correctamente.']);
            break;
    }
}

```

```

        case 'cambiar_estado_usuario':
            $id = $input['id'] ?? '';
            $nuevoEstado = !empty($input['activo']);
            supabaseQuery("usuarios_banco?id=eq." . urlencode($id), "PATCH",
['activo' => $nuevoEstado]);
            echo json_encode(['success' => true, 'message' => 'Estado del usuario
actualizado.']);
            break;

        case 'eliminar_usuario':
            $id = $input['id'] ?? '';
            if (empty($id)) {
                echo json_encode(['success' => false, 'message' => 'ID no
provisto.']);
                exit;
            }
            $res = supabaseQuery("usuarios_banco?id=eq." . urlencode($id),
"DELETE");
            echo json_encode(['success' => true, 'message' => 'Registro eliminado
con éxito.']);
            break;

        // =====
        // 2. GESTIÓN DE TERMINALES POSNET
        // =====
        case 'listar_posnets':
            $res =
supabaseQuery("usuarios_banco?rol=eq.POSNET&select=*&order=id.asc", "GET");
            echo json_encode(['success' => true, 'data' => is_array($res) ? $res :
[]]);
            break;

        case 'guardar_posnet':
            $id = !empty($input['id']) ? trim($input['id']) : null;
            $payload = [
                'usuario' => trim($input['usuario_posnet'] ?? ''),
                'nombre' => trim($input['nombre_stand'] ?? ''),
                'rol' => 'POSNET',
                'activo' => true
            ];

            if (!empty($input['password_posnet'])) {
                $payload['password_hash'] = trim($input['password_posnet']);
            }

            if ($id) {
                $res = supabaseQuery("usuarios_banco?id=eq." . urlencode($id),
"PATCH", $payload);
            } else {
                $payload['monto_acumulado'] = 0;
            }

```

```

        $payload['cant_transacciones'] = 0;
        $payload['puede_retirar'] = false;
        $res = supabaseQuery("usuarios_banco", "POST", $payload);
    }

    if (isset($res['message']) || isset($res['error'])) {
        $errorMsg = $res['message'] ?? $res['error'] ?? 'Error
desconocido';
        echo json_encode(['success' => false, 'message' => 'Error de
Supabase: ' . $errorMsg]);
        exit;
    }

    echo json_encode(['success' => true, 'message' => 'Terminal POSNET
guardada correctamente.']);
    break;

    case 'cambiar_estado_posnet':
        $id = $input['id'] ?? '';
        $nuevoEstado = !empty($input['activo']);
        supabaseQuery("usuarios_banco?id=eq." . urlencode($id), "PATCH",
['activo' => $nuevoEstado]);
        echo json_encode(['success' => true, 'message' => 'Estado del POSNET
actualizado.']);
        break;

    // =====
    // 3. GESTIÓN DE CURSOS
    // =====
    case 'listar_cursos':
        $res = supabaseQuery("cursos?select=*&order=nombre.asc", "GET");
        echo json_encode(['success' => true, 'data' => is_array($res) ? $res :
[]]);
        break;

    case 'guardar_curso':
        $nombre = trim($input['nombre'] ?? '');
        if (empty($nombre)) throw new Exception("El nombre del curso es
obligatorio.");
        supabaseQuery("cursos", "POST", ['nombre' => $nombre]);
        echo json_encode(['success' => true, 'message' => 'Curso agregado
correctamente.']);
        break;

    case 'eliminar_curso':
        $id = $input['id'] ?? '';
        supabaseQuery("cursos?id=eq." . urlencode($id), "DELETE");
        echo json_encode(['success' => true, 'message' => 'Curso eliminado.']);
        break;

```



```

// =====
// 4. IMPORTACIÓN MASIVA DESDE CSV (CORREGIDO)
// =====
case 'importar_csv':
    if (!isset($_FILES['fileCsv']) || $_FILES['fileCsv']['error'] !==
UPLOAD_ERR_OK) {
        throw new Exception("Error al subir el archivo CSV.");
    }

    $fileHandle = fopen($_FILES['fileCsv']['tmp_name'], 'r');
    if (!$fileHandle) {
        throw new Exception("No se pudo abrir el archivo CSV.");
    }

    // Eliminar BOM de UTF-8 si estuviera presente
    $bom = fread($fileHandle, 3);
    if ($bom !== "\xEF\xBB\xBF") {
        rewind($fileHandle);
    }

    // Omitir la fila de encabezados
    fgetcsv($fileHandle, 1000, ",", "");

    $insertados = 0;
    $errores = [];
    $filaNum = 1;

    while (($row = fgetcsv($fileHandle, 1000, ",")) !== FALSE) {
        $filaNum++;

        // Ignorar filas totalmente vacías
        if (empty($row) || (count($row) === 1 && trim($row[0]) === '')) {
            continue;
        }

        $dni = trim($row[0] ?? '');
        if (empty($dni)) {
            continue; // Saltar si no tiene DNI
        }

        $fotoUrl = isset($row[4]) && trim($row[4]) !== '' ? trim($row[4])
: null;

        $codigoQr = isset($row[5]) && trim($row[5]) !== '' ? trim($row[5])
: null;

        $payload = [
            'dni' => $dni,
            'nombre_apellido' => trim($row[1] ?? ''),
            'curso' => trim($row[2] ?? ''),

```

```

        'saldo'                => is_numeric($row[3] ?? null) ?
floatval($row[3]) : 0.00,
        'foto_url'            => $fotoUrl,
        'codigo_qr'           => $codigoQr,
        'registrado'           => false
    ];

    // Ejecutar la petición a Supabase
    $res = supabaseQuery("alumnos", "POST", $payload);

    // Verificar si Supabase devolvió un error de inserción
    if (isset($res['message']) || isset($res['error']) ||
isset($res['hint'])) {
        $msgErr = $res['message'] ?? ($res['error'] ?? 'Error
desconocido');
        $errores[] = "Fila $filaNum (DNI $dni): $msgErr";
    } else {
        $insertados++;
    }
}

fclose($fileHandle);

if (count($errores) > 0 && $insertados === 0) {
    echo json_encode([
        'success' => false,
        'message' => "No se pudo insertar ningún alumno. Detalle de
error: " . $errores[0]
    ]);
} else if (count($errores) > 0) {
    echo json_encode([
        'success' => true,
        'message' => "Se insertaron $insertados alumnos. Ocurrieron
errores en algunas filas: " . implode(" | ", $errores)
    ]);
} else {
    echo json_encode([
        'success' => true,
        'message' => "Se importaron $insertados alumnos correctamente
en la base de datos."
    ]);
}
break;
// =====
// 5. MANTENIMIENTO Y RESET DE BD
// =====
case 'reset_bd':
    $qrIngresado = trim($input['qr_code'] ?? '');

    if (defined('QR_MAESTRO_ADMIN') && $qrIngresado !== QR_MAESTRO_ADMIN) {

```

```

        throw new Exception("QR Maestro de confirmación inválido.");
    }

    $opciones = $input['opciones'] ?? [];

    if (!empty($opciones['reset_acumulados'])) {
        supabaseQuery("usuarios_banco?id=gt.0", "PATCH", ['monto_acumulado'
=> 0, 'cant_transacciones' => 0]);
    }
    if (!empty($opciones['vaciar_transacciones'])) {
        supabaseQuery("transacciones?id=gt.0", "DELETE");
    }
    if (!empty($opciones['vaciar_logs'])) {
        supabaseQuery("logs_sistema?id=gt.0", "DELETE");
        supabaseQuery("logs_auditoria?id=gt.0", "DELETE");
    }
    if (!empty($opciones['borrar_alumnos_no_registrados'])) {
        supabaseQuery("alumnos?registrado=eq.false", "DELETE");
    }
    if (!empty($opciones['vaciar_alumnos_todos'])) {
        supabaseQuery("alumnos?id=neq.00000000-0000-0000-0000-000000000000", "DELETE");
    }

    echo json_encode(['success' => true, 'message' => "Mantenimiento
realizado con éxito."]);
    break;

    default:
        throw new Exception("Acción no válida.");
    }
} catch (Exception $e) {
    echo json_encode(['success' => false, 'message' => $e->getMessage()]);
}

```